Equivalence Checking between Programs

Chandan Karfa



ভাৰতীয় প্ৰযুক্তিবিদ্যা প্ৰতিষ্ঠান গুৱাহাটী
भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI

Outline

1. Background
2. Translation Validation

Outline

1 Background

2 Translation Validation

Background

Program: An organized list of instructions that, when executed, causes the computer to behave in a predetermined manner.
(source: Venit et al., Prelude to Programming: Concepts and Design)
Even if we write (syntactically and semantically) correct programs, they may not be "good" programs.

Example of code optimization

```
for(i = 1; i <= N; i++) {
  if( x > 10 )
    a[i] = b[i] + c[i];
  else
    a[i] = b[i] - c[i];
}
```

```
if( x > 10 )
  for(i = 1; i <= N; i++)
    a[i] = b[i] + c[i];
else
  for(i = 1; i <= N; i++)
    a[i] = b[i] - c[i];
```

Objectives of code optimization:

- To speed-up the computation
- To use less resource, eg. memory, power, etc.

So, we need a **compiler**.

Background

Program: An organized list of instructions that, when executed, causes the computer to behave in a predetermined manner.
(source: Venit et al., Prelude to Programming: Concepts and Design)
Even if we write (syntactically and semantically) correct programs, they may not be "good" programs.

Example of code optimization

```
for(i = 1; i <= N; i++) {
  if( x > 10 )
    a[i] = b[i] + c[i];
  else
    a[i] = b[i] - c[i];
}
```

```
if( x > 10 )
  for(i = 1; i <= N; i++)
    a[i] = b[i] + c[i];
else
  for(i = 1; i <= N; i++)
    a[i] = b[i] - c[i];
```

Objectives of code optimization:

- To speed-up the computation
- To use less resource, eg. memory, power, etc.

So, we need a **compiler**.

Background

Program: An organized list of instructions that, when executed, causes the computer to behave in a predetermined manner.
(source: Venit et al., Prelude to Programming: Concepts and Design)
Even if we write (syntactically and semantically) correct programs, they may not be "good" programs.

Example of code optimization

```
for(i = 1; i <= N; i++) {
  if( x > 10 )
    a[i] = b[i] + c[i];
  else
    a[i] = b[i] - c[i];
}
```

```
if( x > 10 )
  for(i = 1; i <= N; i++)
    a[i] = b[i] + c[i];
else
  for(i = 1; i <= N; i++)
    a[i] = b[i] - c[i];
```

Objectives of code optimization:

- To speed-up the computation
- To use less resource, eg. memory, power, etc.

So, we need a **compiler**.

Background

Program: An organized list of instructions that, when executed, causes the computer to behave in a predetermined manner.
(source: Venit et al., Prelude to Programming: Concepts and Design)
Even if we write (syntactically and semantically) correct programs, they may not be "good" programs.

Example of code optimization

```
for(i = 1; i <= N; i++) {
  if( x > 10 )
    a[i] = b[i] + c[i];
  else
    a[i] = b[i] - c[i];
}
```

```
if( x > 10 )
  for(i = 1; i <= N; i++)
    a[i] = b[i] + c[i];
else
  for(i = 1; i <= N; i++)
    a[i] = b[i] - c[i];
```

Objectives of code optimization:

- To speed-up the computation
- To use less resource, eg. memory, power, etc.

So, we need a **compiler**.

Background

Program: An organized list of instructions that, when executed, causes the computer to behave in a predetermined manner.
(source: Venit et al., Prelude to Programming: Concepts and Design)
Even if we write (syntactically and semantically) correct programs, they may not be "good" programs.

Example of code optimization

```
for(i = 1; i <= N; i++) {
  if( x > 10 )
    a[i] = b[i] + c[i];
  else
    a[i] = b[i] - c[i];
}
```

```
if( x > 10 )
  for(i = 1; i <= N; i++)
    a[i] = b[i] + c[i];
else
  for(i = 1; i <= N; i++)
    a[i] = b[i] - c[i];
```

Objectives of code optimization:

- To speed-up the computation
- To use less resource, eg. memory, power, etc.

So, we need a **compiler**.

Can you trust your compiler?

- Present day compilers are sophisticated enough to carry out many intricate transformations
- A human expert may also play the role of a compiler
- Compiler transformations, even if theoretically correct, may be buggy because of wrong coding

Erroneous loop interchange

```
sum = 0;
for (j=0; j<N; j++) {
  for (i=0; i<N; i++) {
    sum = sum + a[i][j];
} }
```

```
sum = 0;
for (i=0; i<N; i++) {
  for (j=0; j<=N; j++) {
    sum = sum + a[i][j];
} } // a[i][N] gets accessed
```

Program: An organized list of instructions that, when executed, causes the computer to behave in a predetermined manner.

A faulty compiler can alter the meaning of a program.

Can you trust your compiler?

- Present day compilers are sophisticated enough to carry out many intricate transformations
- A human expert may also play the role of a compiler
- Compiler transformations, even if theoretically correct, may be buggy because of wrong coding

Erroneous loop interchange

```
sum = 0;
for (j=0; j<N; j++) {
  for (i=0; i<N; i++) {
    sum = sum + a[i][j];
} }
```

```
sum = 0;
for (i=0; i<N; i++) {
  for (j=0; j<=N; j++) {
    sum = sum + a[i][j];
} } // a[i][N] gets accessed
```

Program: An organized list of instructions that, when executed, causes the computer to behave in a predetermined manner.

A faulty compiler can alter the meaning of a program.

Untrusted compilers is a reality

gcc – Frequently Reported Bugs

There are many reasons why a reported bug doesn't get fixed. It might be difficult to fix, or fixing it might break compatibility. Often, reports get a low priority when there is a simple work-around. In particular, bugs caused by invalid code have a simple work-around: fix the code.
(source: `http://gcc.gnu.org/bugs/#known`)

What is the remedy?

Verified Compiler

All optimized programs will be correct *by construction*
Example: CompCert, INRIA
Limitations

- Very hard to formally verify all passes of a compiler
- Undecidability of the general problem restricts the scope of the input language

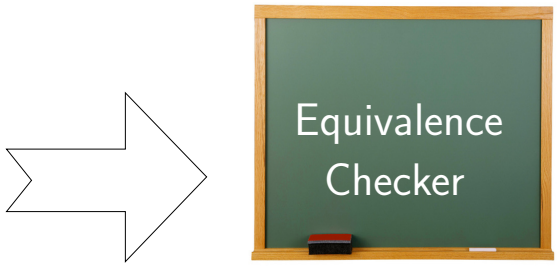
Translation Validation

Each individual translation is followed by a validation phase which verifies that the target code produced correctly implements the source code
(This is what we do, i.e., equivalence checking of programs)

Translation Validation



Specification





Implementation



Compiler

Equivalence of finite automata

Definition

Two finite automata (FA) over Σ are equivalent if they accept the same set of strings over Σ.

By corollary, when two FA are not equivalent, with both the FA starting from the initial state, there must be a string w for which one of the two FA reaches a final/accepting state, while the other FA does not.

Finite automata based verification strategy

Limitations

- State based representation not powerful enough to capture execution of programs
- Practical programs have a finite set of states, but that does not permit efficient analysis of programs
- Computed values need to stored
- Separation of control states and data states is needed

So, we need some kind of control and data-flow graph (CDFG) to reason about the programs.

Finite automata based verification strategy

Limitations

- State based representation not powerful enough to capture execution of programs
- Practical programs have a finite set of states, but that does not permit efficient analysis of programs
- Computed values need to stored
- Separation of control states and data states is needed

So, we need some kind of control and data-flow graph (CDFG) to reason about the programs.

How to check equivalence of programs?

The general problem is undecidable.

McCarthy 91 function

```
int M ( int n ) {
if (n > 100)
  return (n - 10);
else
  return M( M (n + 11) );
}
```

```
int M ( int n ) {
if (n > 100)
  return (n - 10);
else
  return 91;
}
```

Comparing two programs in *totality* is impossible – we should break them into *smaller* chunks.

Granularity of the chunks

Instruction level

```
x = a + b;
y = x - a;
z = y + b;
```

```
x = a + b;
y = b;
z = 2 * b;
```

Granularity of the chunks

Instruction level

```
x = a + b;
y = x - a;
z = y + b;
```

✓

```
x = a + b;
y = b;
z = 2 * b;
```

✓

Granularity of the chunks

Instruction level

```
x = a + b;  ✓          x = a + b;  ✓
y = x - a;  ×          y = b;      ×
z = y + b;             z = 2 * b;
```

So, instruction level checking can be misleading – let's try at basic block level.

Granularity of the chunks (contd.)

Basic Block level

```
x = a + b;
y = x - a;
z = y + b;
do {
  v = v + x;
  w = y * z;
} while( c1 );
```

```
x = a + b;
y = b;
z = 2 * b;
do {
  v = v + x;
} while( c1 );
w = y * z;
```

Granularity of the chunks (contd.)

Basic Block level

```
x = a + b; ✓
y = x - a; ✓
z = y + b; ✓
do {
  v = v + x;
  w = y * z;
} while( c1 );
```

```
x = a + b; ✓
y = b;     ✓
z = 2 * b; ✓
do {
  v = v + x;
} while( c1 );
w = y * z;
```

Granularity of the chunks (contd.)

Basic Block level

```
x = a + b; ✓
y = x - a; ✓
z = y + b; ✓
do {
  v = v + x; ×
  w = y * z; ×
} while( c1 );
```

```
x = a + b; ✓
y = b;     ✓
z = 2 * b; ✓
do {
  v = v + x; ×
} while( c1 );
w = y * z;
```

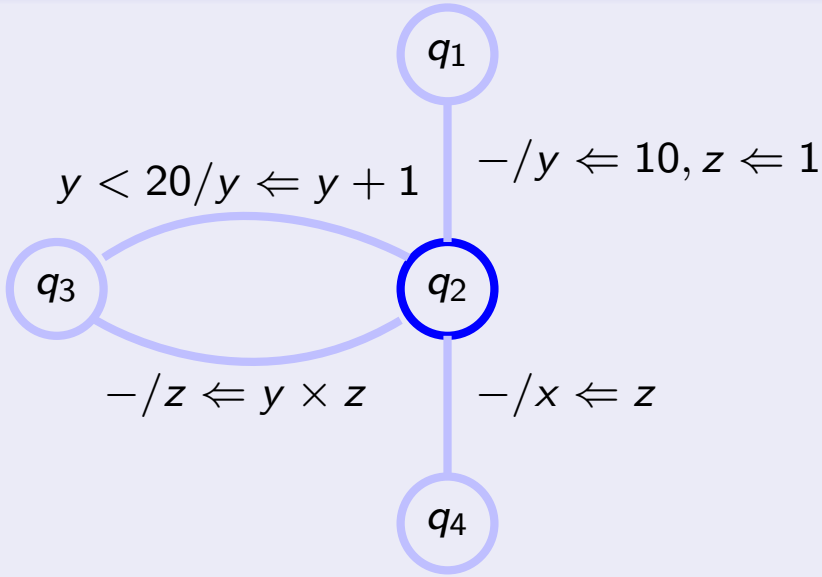
So, checking individual basic blocks is not enough.

Program as a composition of finite paths

A program can have *infinite* number of computations, a single computation can be *indefinitely long* — **cut loops**.

Representing a program using CDFG

```
y := 10;
z := 1;
while ( y < 20 ) {
  y := y + 1;
  z := y × z;
}
x := z;
```



All computations of the program can be viewed as a concatenation of paths.

Example: $p_1.p_3$, $p_1.p_2.p_3$, $p_1.p_2.p_2.p_3$, $p_1.(p_2)^*.p_3$

Outline

1 Background

2 Translation Validation

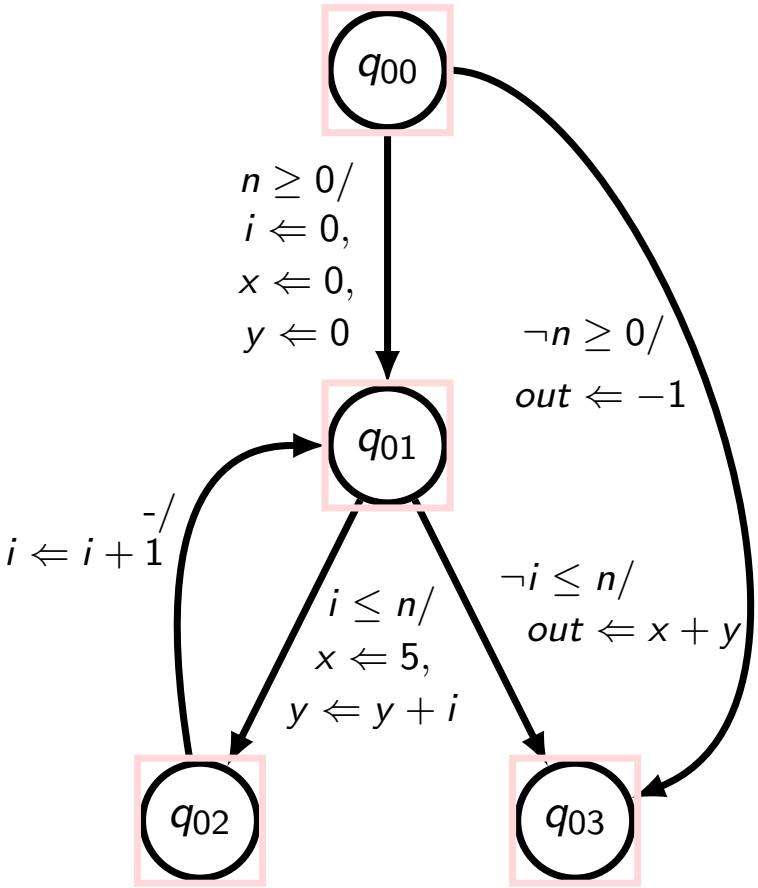
Finite State Machine with Datapath (FSMD)

FSMDs effectively capture both the control flow and the associated data processing of a behaviour.

The FSMD model is a seven tuple $F = \langle Q, q_0, I, V, O, f, h \rangle$:

- Q: Finite set of control states
- q_0: Reset state, i.e. $q_0 \in Q$
- I: Set of input variables
- V: Set of storage variables
- O: Set of output variables
- f: State transition function, i.e. $Q \times 2^S \rightarrow Q$
- h: Update function of the output and the storage variables, i.e. $Q \times 2^S \rightarrow U$
  - U represents a set of storage or output assignments
  - S is a set of arithmetic relations between arithmetic expressions

Finite State Machine with Datapath (FSMD)

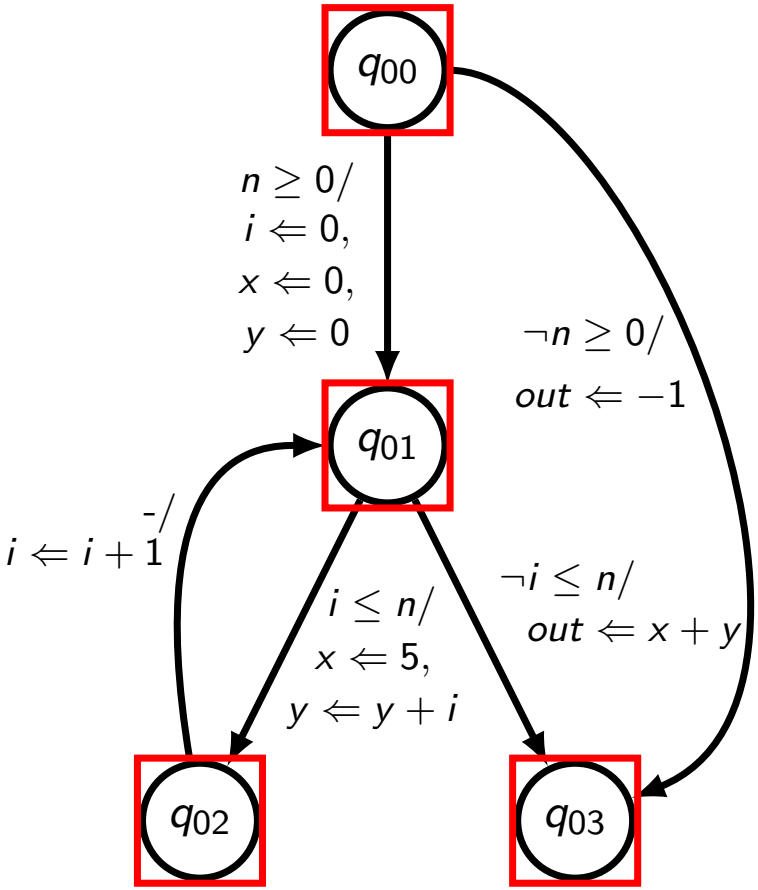


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}$ $q_0 = q_{00}$

$I = \{n\}$ $O = \{out\}$ $V = \{x, y, i\}$

Finite State Machine with Datapath (FSMD)

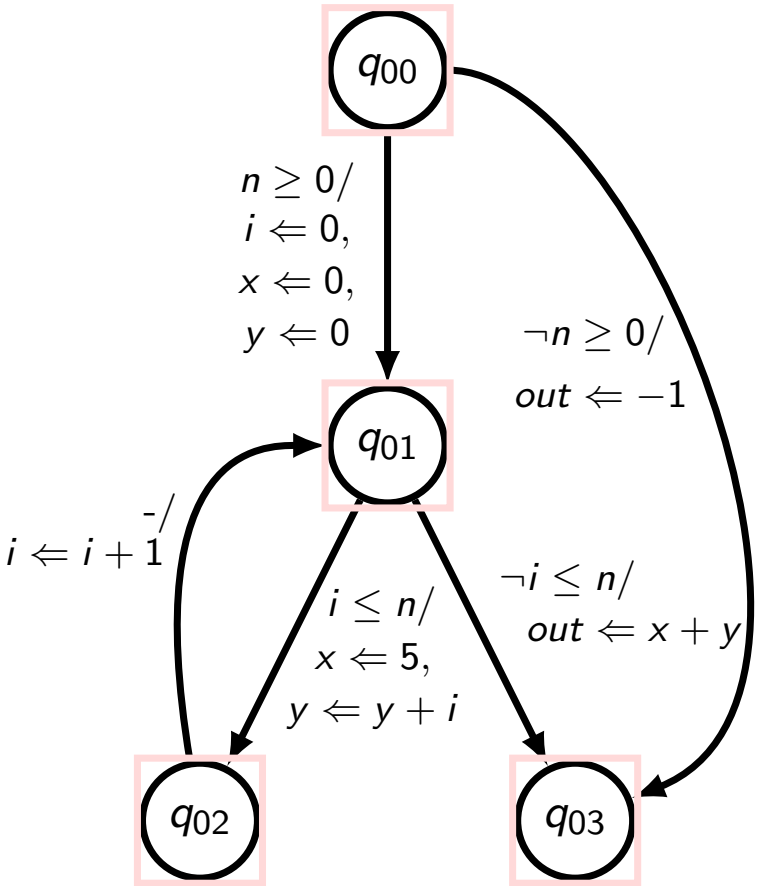


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}$ $q_0 = q_{00}$

$I = \{n\}$ $O = \{out\}$ $V = \{x, y, i\}$

Finite State Machine with Datapath (FSMD)

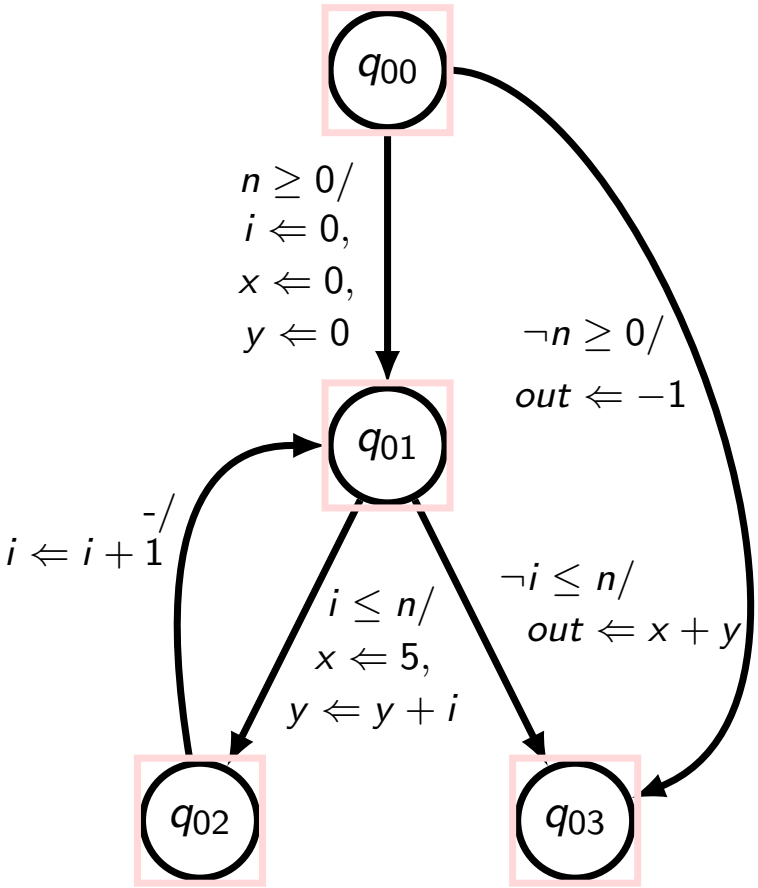


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}\ q_0 = q_{00}$$

$$I = \{n\}\ O = \{out\}\ V = \{x, y, i\}$$

Finite State Machine with Datapath (FSMD)

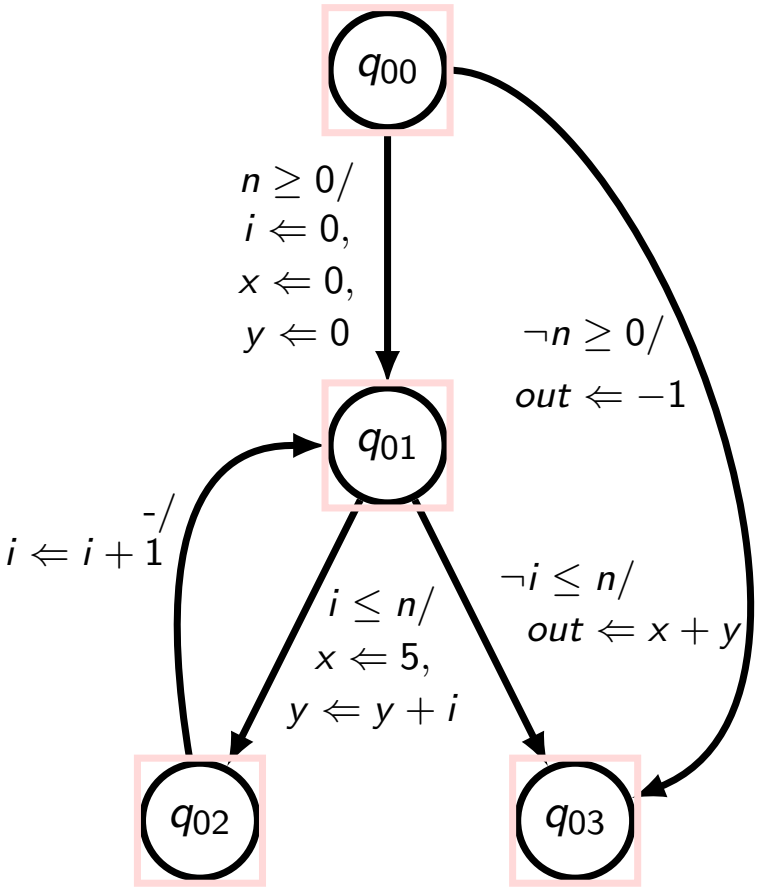


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}\ q_0 = q_{00}$$

$$I = \{n\}\ O = \{out\}\ V = \{x, y, i\}$$

Finite State Machine with Datapath (FSMD)

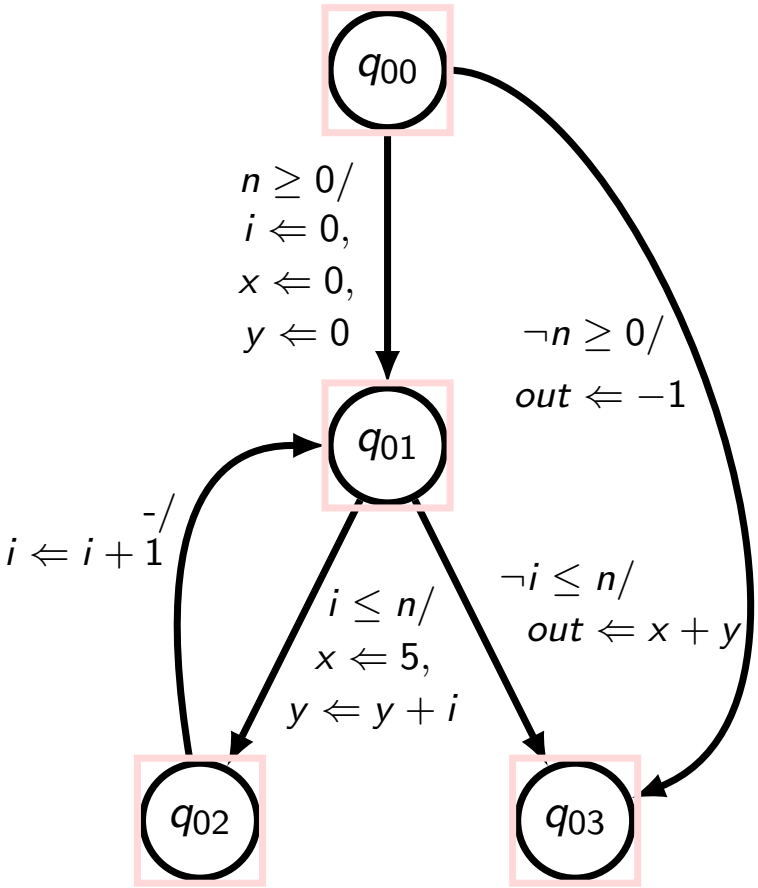


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}\ q_0 = q_{00}$$

$$I = \{n\}\ O = \{out\}\ V = \{x, y, i\}$$

Finite State Machine with Datapath (FSMD)

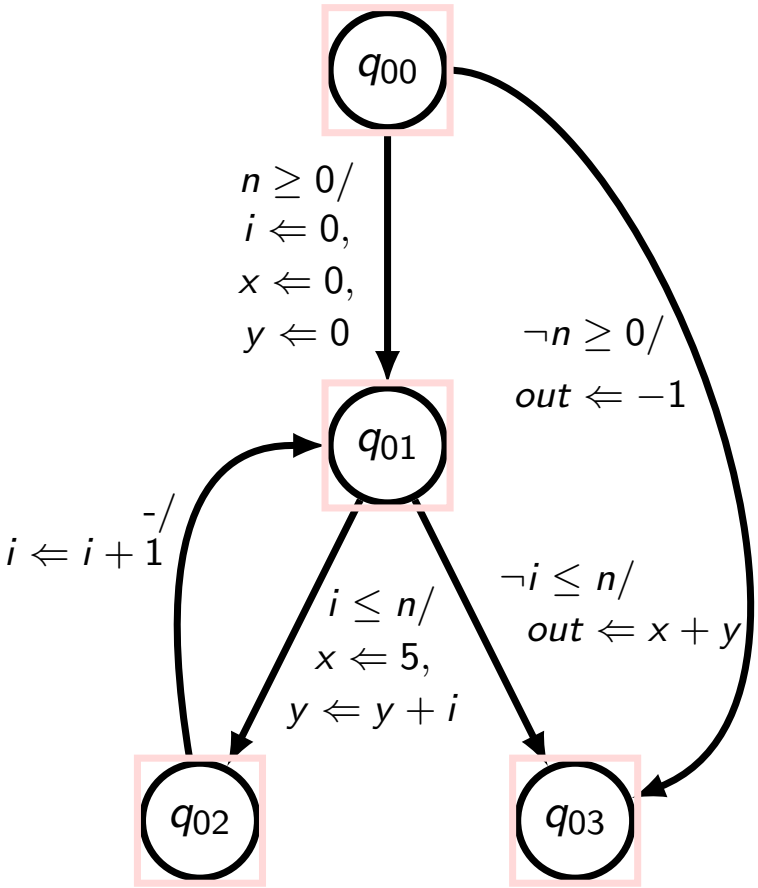


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}\ q_0 = q_{00}$$

$$I = \{n\}\ O = \{out\}\ V = \{x, y, i\}$$

Finite State Machine with Datapath (FSMD)

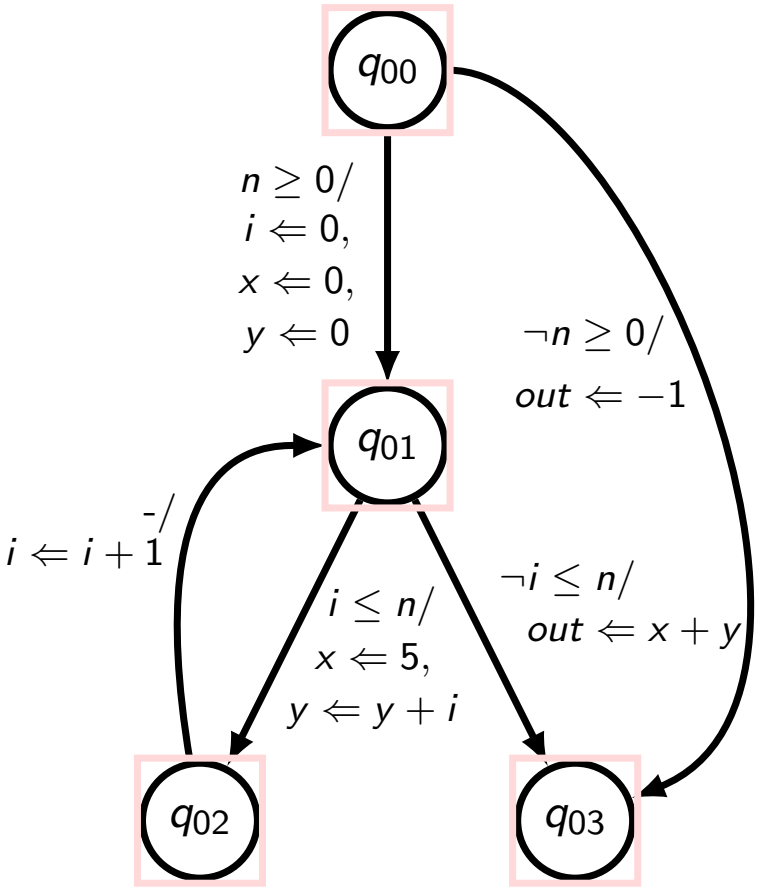


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}\ q_0 = q_{00}$$

$$I = \{n\}\ O = \{out\}\ V = \{x, y, i\}$$

Finite State Machine with Datapath (FSMD)

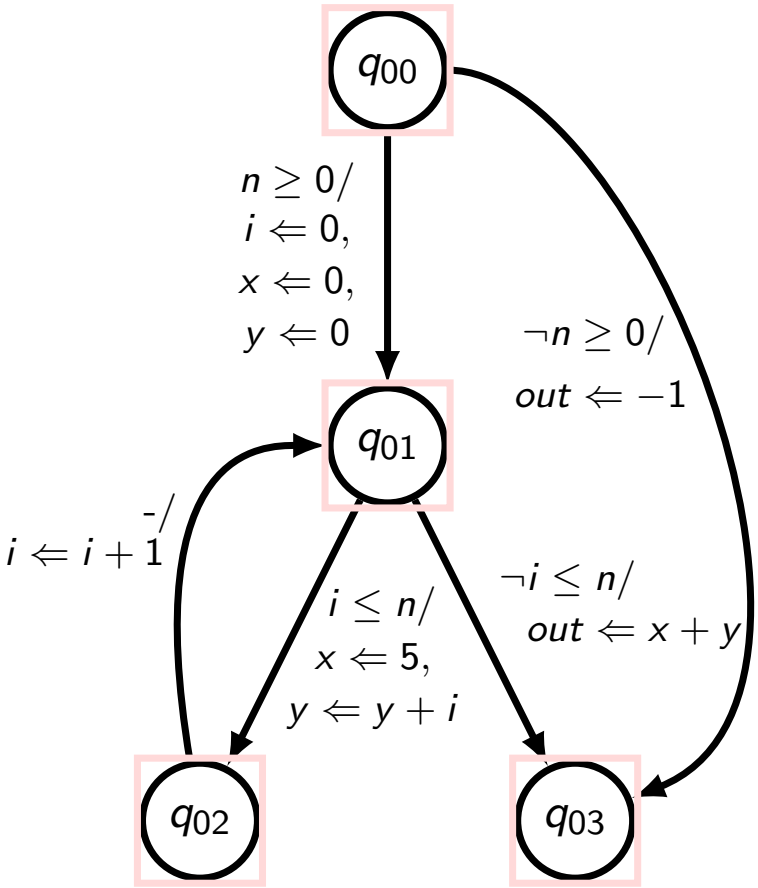


1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}\ q_0 = q_{00}$$

$$I = \{n\}\ O = \{out\}\ V = \{x, y, i\}$$

Finite State Machine with Datapath (FSMD)



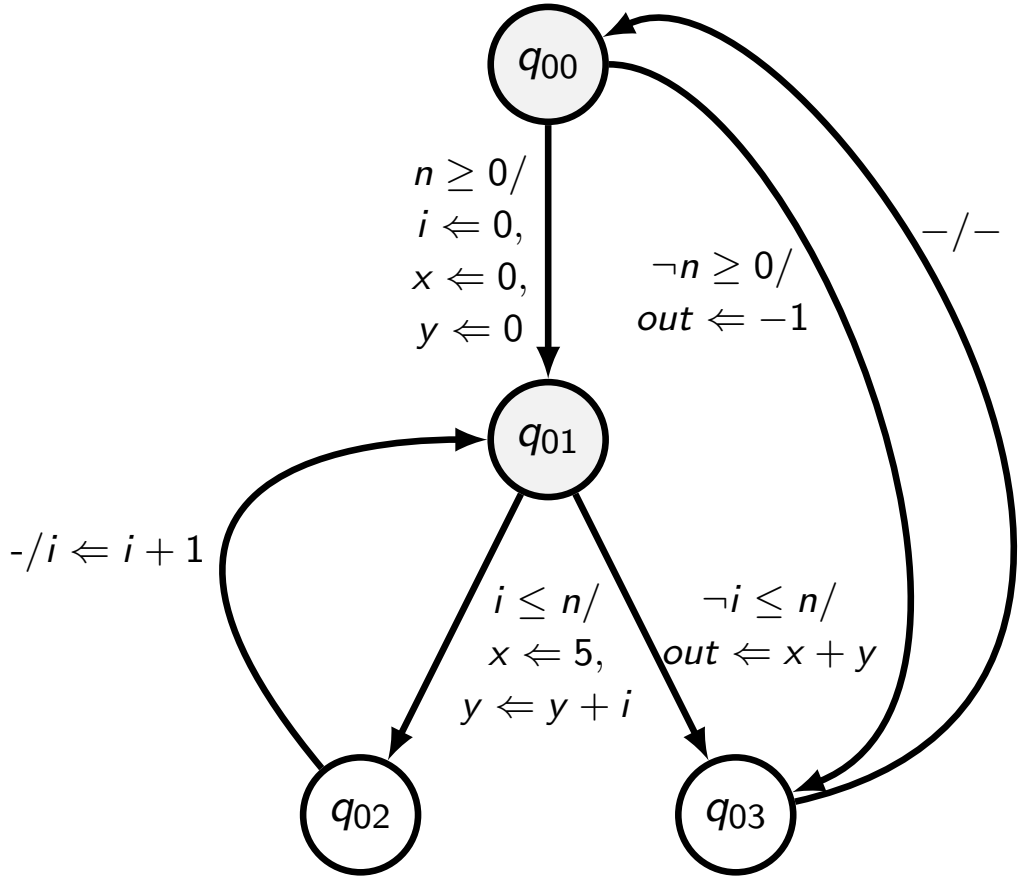
1. Q finite set of states
2. $q_0 \in Q$ reset (initial) state
3. I finite set of input variables
4. O finite set of output variables
5. V finite set of storage variables
6. f : state transition function
7. h : update function
8. Deterministic model.

$$Q == \{q_{00}, q_{01}, q_{02}, q_{03}\}\ q_0 = q_{00}$$

$$I = \{n\}\ O = \{out\}\ V = \{x, y, i\}$$

Representing a program using FSMD

```
if(n≥0){
x=0,y=0;
for(i=0;i≤n;i++){
x=5;
y=y+i;}
out=x+y;} else
out=-1;
```



Path and Transformation along a Path

- **Path α from q_i to q_j:**

$$\langle q_i = q_1 \xrightarrow{\bar{C}_1} q_2 \xrightarrow{\bar{C}_2} \cdots \xrightarrow{\bar{C}_{n-1}} q_n = q_j \rangle$$

A finite transition sequence, where all q_i, $1 \leq i \leq n-1$, are distinct; only q_n may be equal to q_1.

- **Condition of execution of a path α (R_α):**
Logical expression over the set $I \cup V$ such that if R_α is satisfied by the data state at the beginning of α, then α is traversed.

- **Data transformation over a path α (r_α):**

$$r_\alpha = \langle S_\alpha, O_\alpha \rangle, \qquad S_\alpha = \langle e_1, e_2, \ldots, e_k \rangle$$

An ordered tuple of algebraic expressions over variables in V. e_i represents the value of variable v_i after execution of the path α. O_α is the output list of the path.

Path Transformation and Equivalence of Two Paths

$$R_\alpha = i \leq 15 \wedge b \bmod 2 = 1 \wedge s+a \geq n$$

$$r_\alpha = \langle \langle a, b, n, i, s, s + a - n \rangle, - \rangle$$

Order of the variables: $\langle a, b, n, i, s \rangle$.
There is no output list for this path.

Equivalence of two paths

If $R_\alpha \equiv R_\beta$ and $r_\alpha = r_\beta$

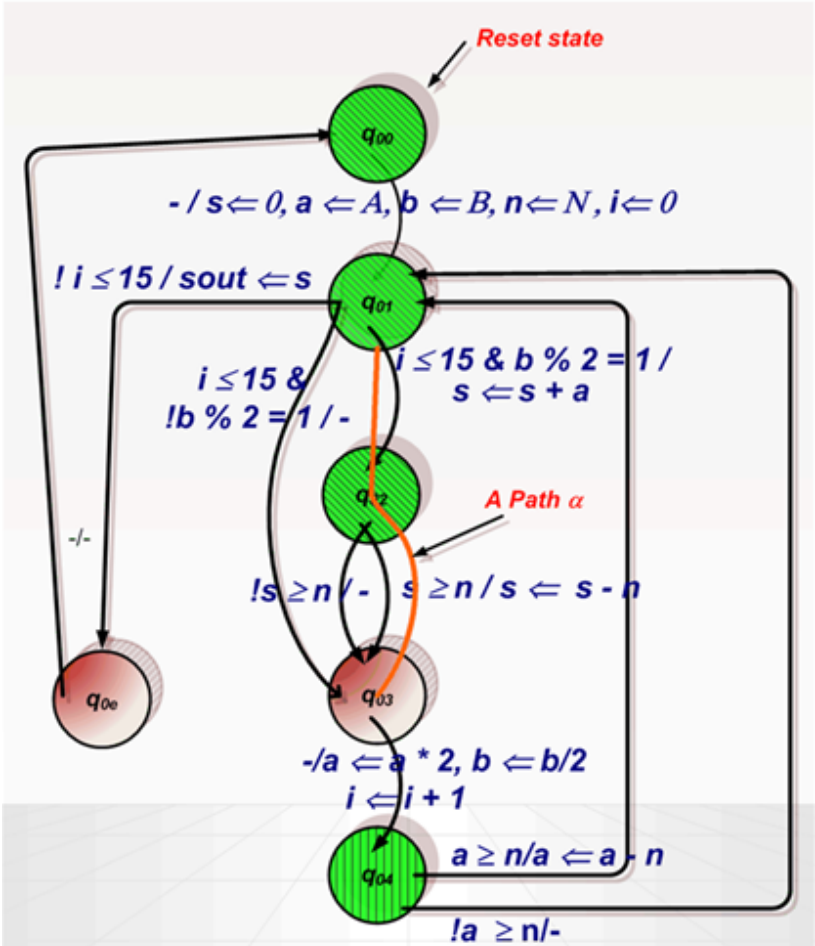


Fig. 1 FSMD of (a*b modn) before scheduling

FSMD

Computation on the FSMDs

- A finite walk from the reset state back to itself without having any intermediary occurrence of reset state.
- Two computations c_1 and c_2 are equivalent if

$$R_{c_1} = R_{c_2} \quad \& \quad r_{c_1} = r_{c_2}.$$

Length and number of computations of an FSMD can both be infinite.

- Computation can be looked upon as a concatenation of paths $[\alpha_1, \alpha_2, \ldots, \alpha_k]$
  where α_1 emanates from and α_k terminates in the reset state of the FSMD and α_i terminates at the start node of α_{i+1} for all $i = 1$ to $k - 1$.

Finite Path Cover and Equivalence of Two FSMDs

Finite Path Cover:

- A set $P = \{p_1, p_2, \ldots, p_k\}$ of FSMD M is said to be a path cover of M iff any computation of M can be looked upon as a concatenation of paths from P.

Equivalence of two FSMDs:

- For any computation c_0 of M_0, there exists a computation c_1 in M_1 and vice-versa.
- In other words, if there exists a finite path cover $P_0 = \{p_{00}, p_{01}, p_{02}, \ldots, p_{0l}\}$ of M_0, for which there exists a set $P_1 = \{p_{10}, p_{11}, p_{12}, \ldots, p_{1l}\}$ of M_1 such that $p_{0i} \cong p_{1i}$ for all $0 \leq i \leq l$ and vice-versa.

Outline of the Verification Method

Outline of the Verification method:

- Let M_0 and M_1 be the FSMDs corresponding to the behaviours before and after of any phase, respectively.

Steps:

1. Find the path cover, P_0, in M_0.
2. For every path in P_0, find an equivalent path in M_1.
3. Repeat Step 1 and 2 with FSMDs interchanged.

Problem with finding Path Cover:

- **Loops** in FSMD.

Solutions

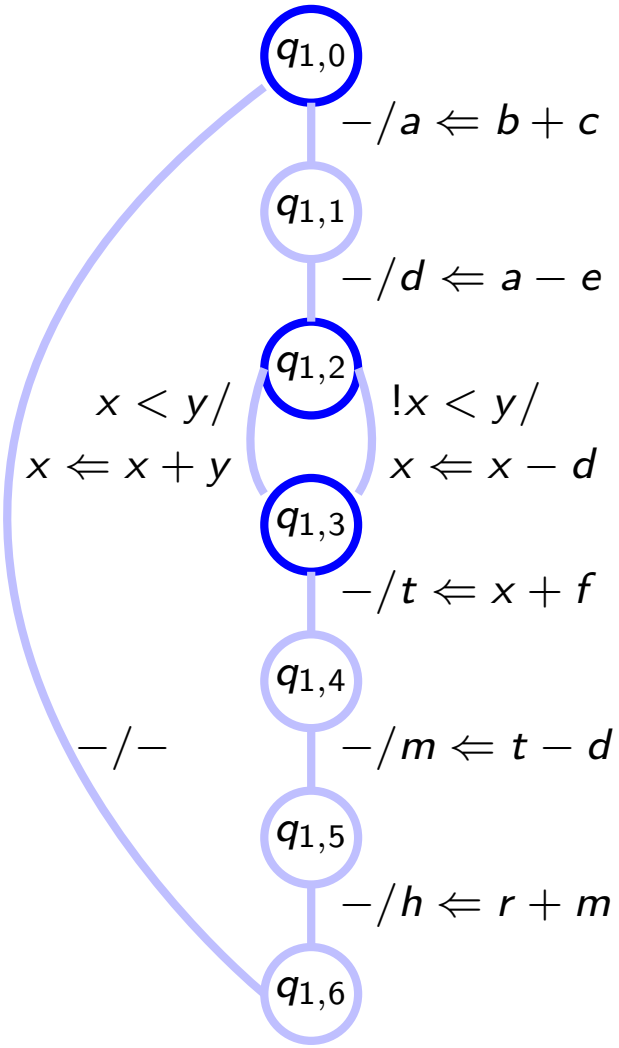
Solutions:

- Based on **Floyd–Hoare's method of program verification**.
- Put cutpoints in FSMD such that each loop is cut at least one cutpoint.
- Set of paths from one cutpoint to another without having any intermediate cutpoint is a path cover.

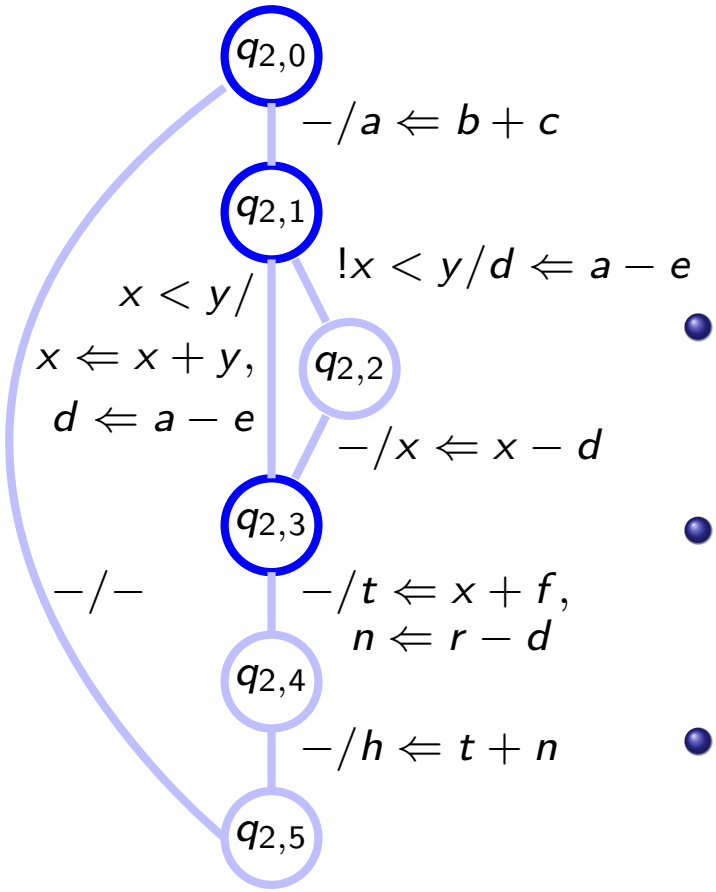
Cutpoints: Defined as follows in our work

1. Reset state.
2. State with more than one outward transition.

Equivalence checking of FSMDs: A basic example



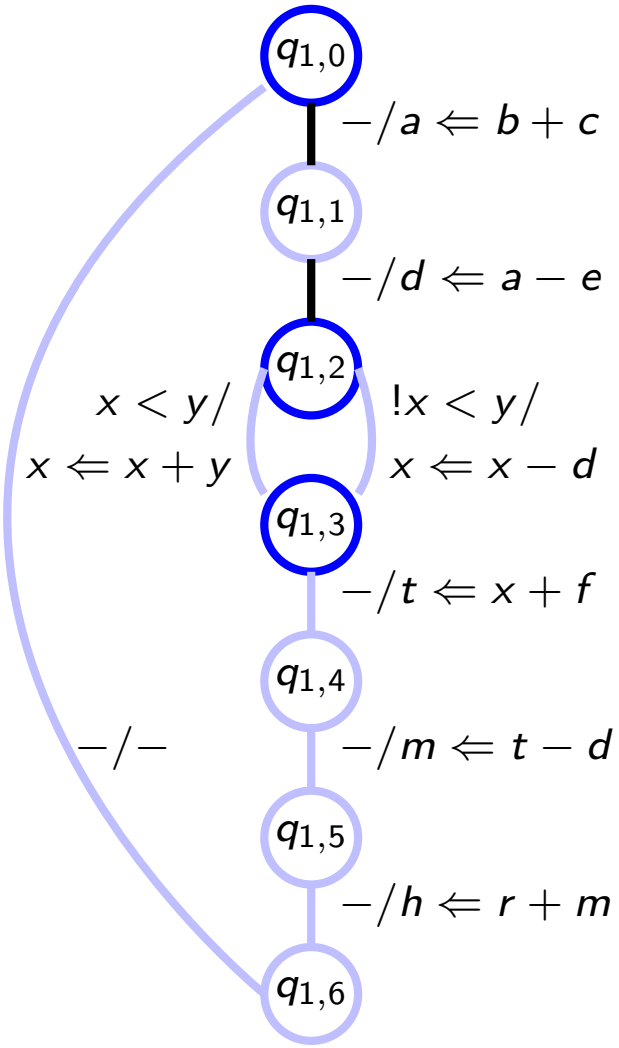
(a) M_1



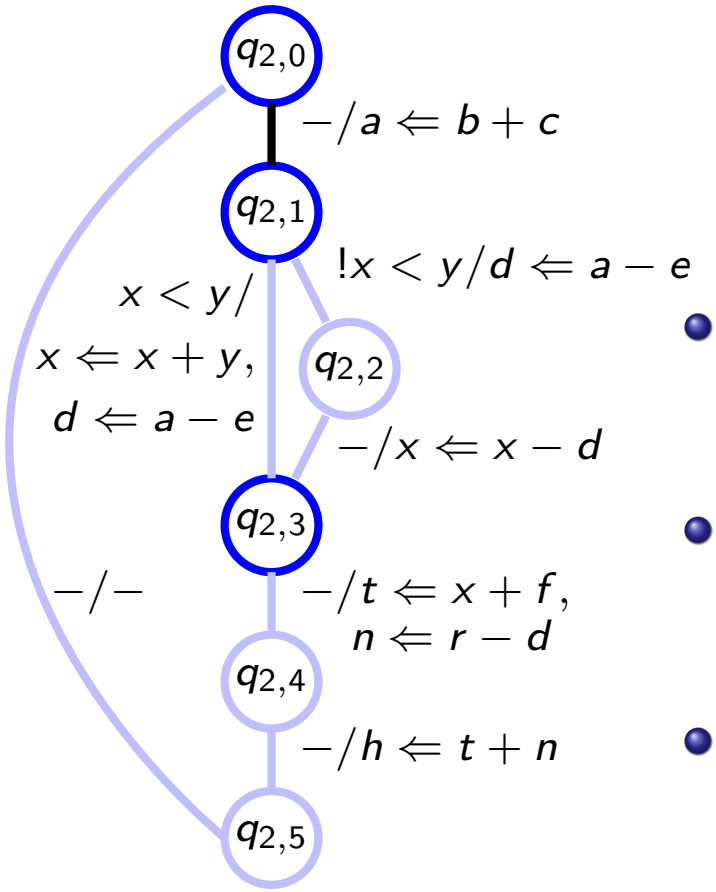
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3},$ $q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



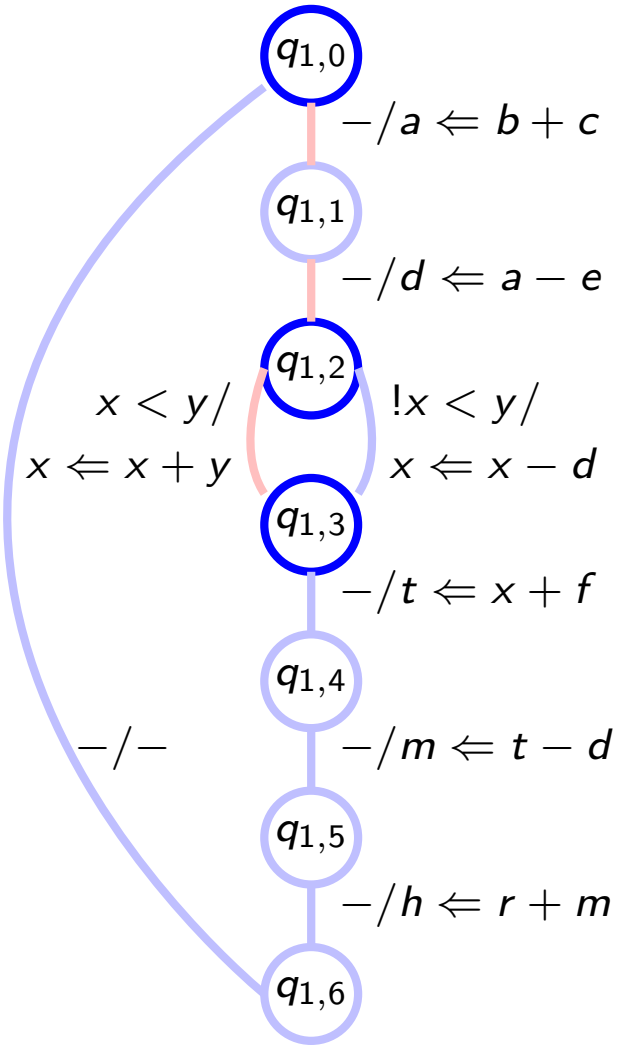
(a) M_1



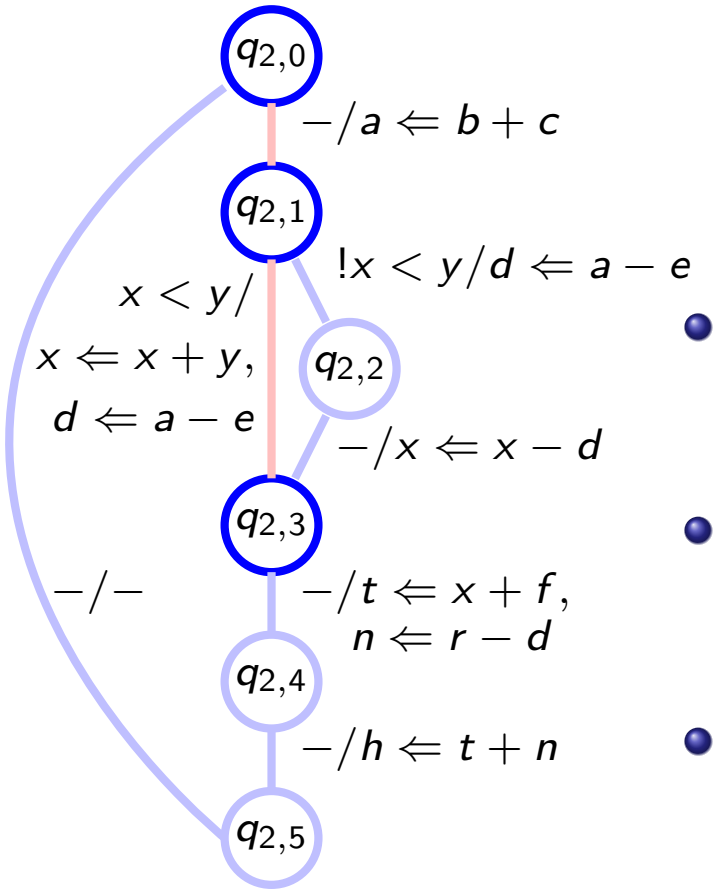
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3},$ $q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



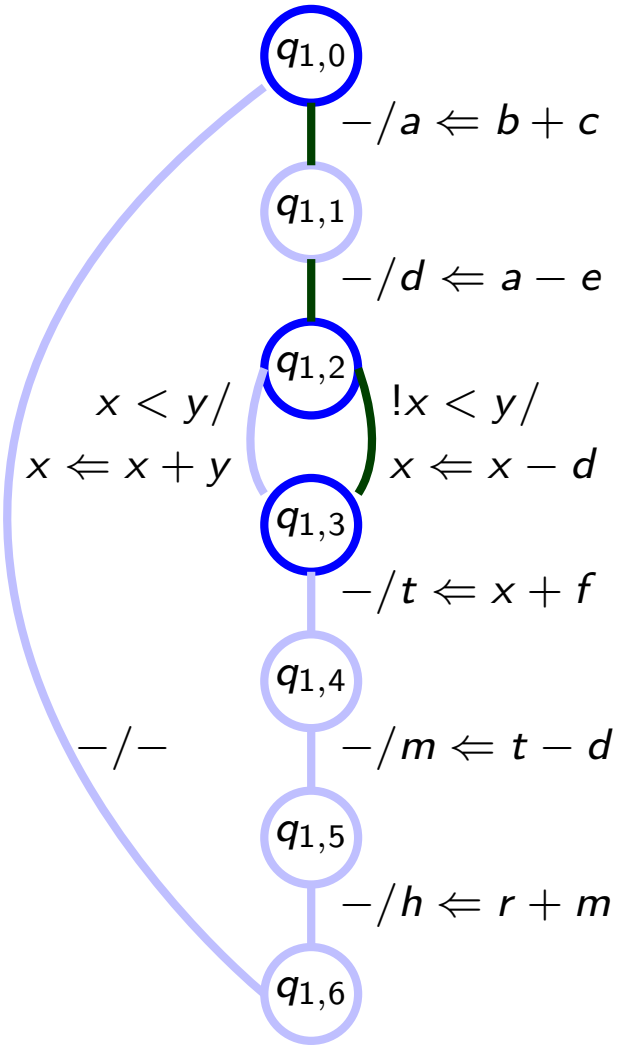
(a) M_1



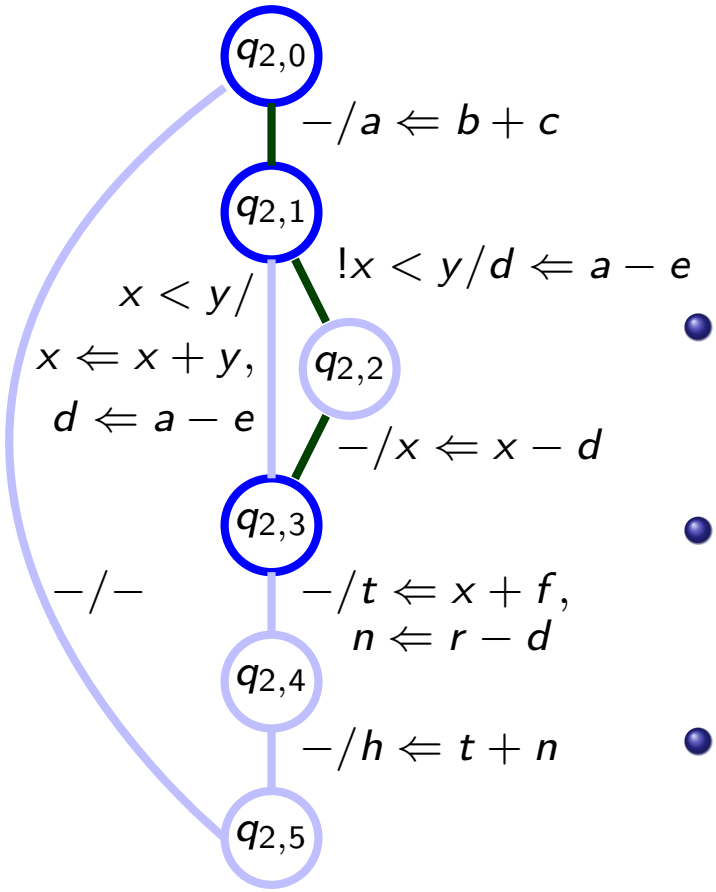
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3},$ $q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



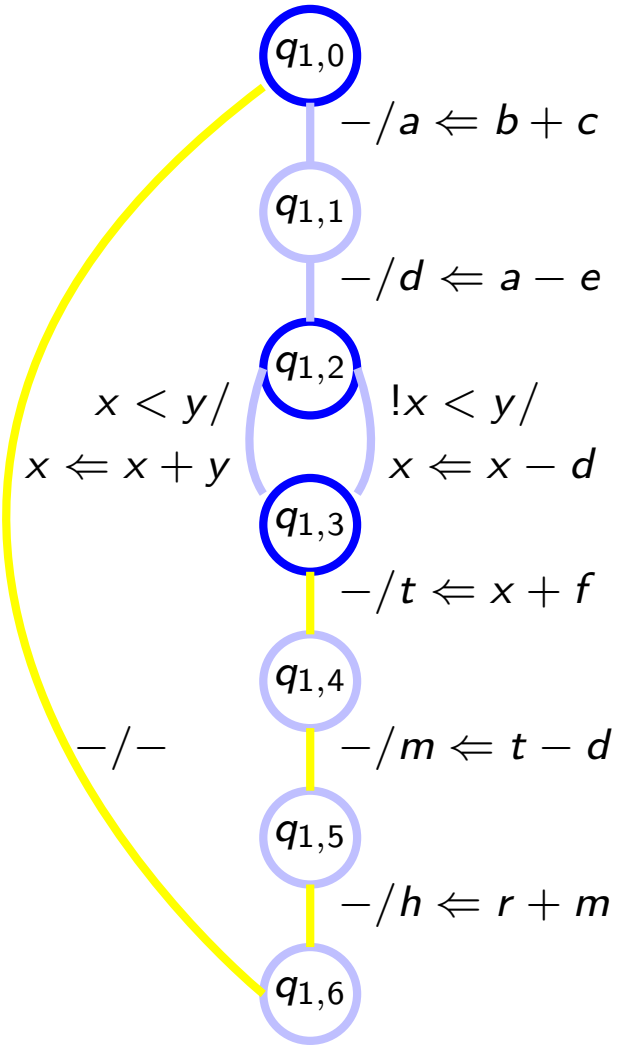
(a) M_1



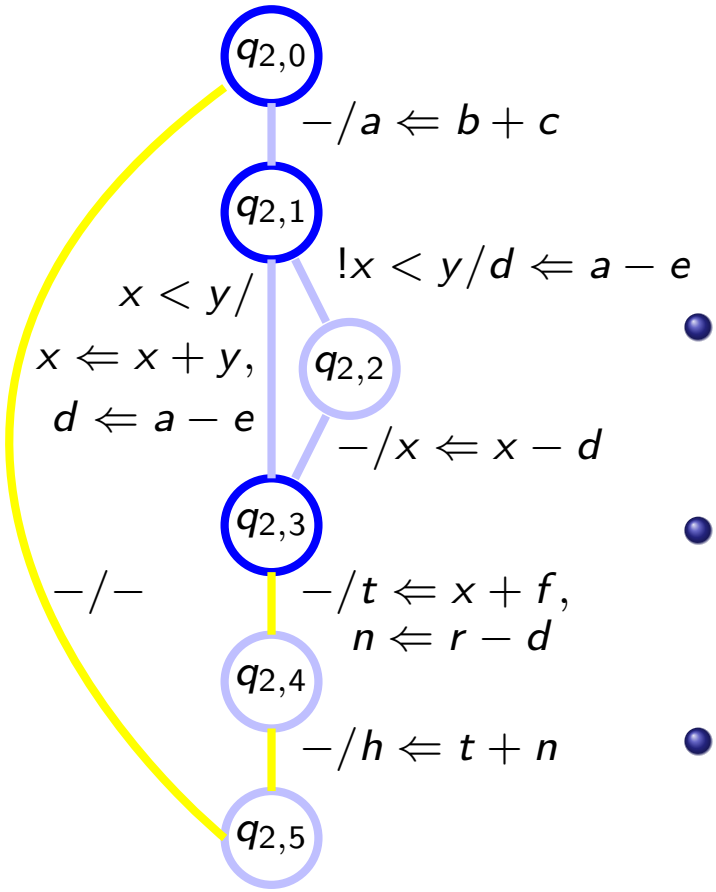
(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \overset{x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{x<y}{\Longrightarrow} q_{2,3}, q_{1,0} \overset{!x<y}{\Longrightarrow} q_{1,3} \simeq q_{2,0} \overset{!x<y}{\Longrightarrow} q_{2,3}, q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

Equivalence checking of FSMDs: A basic example



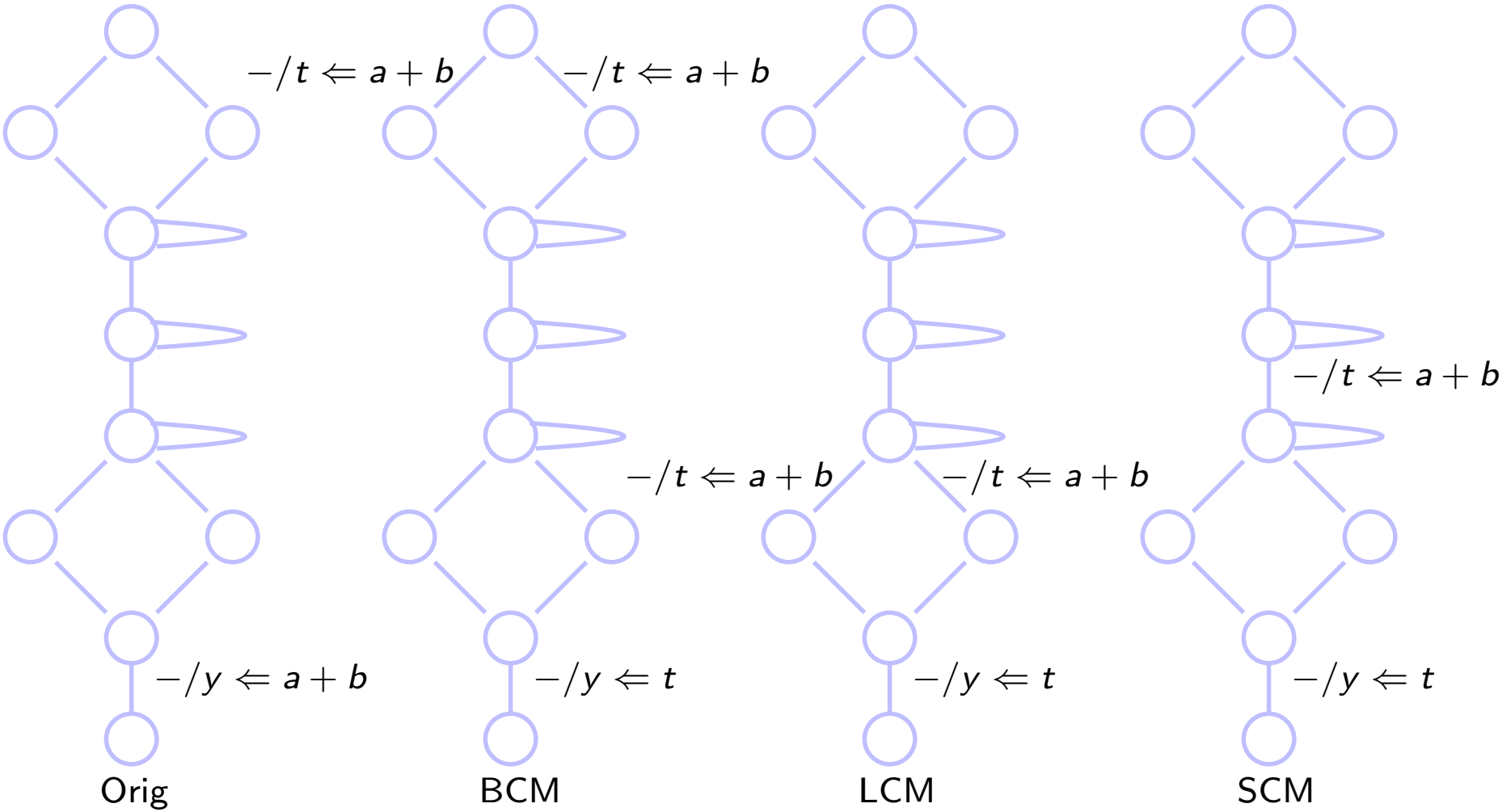
(a) M_1



(b) M_2

- Two Finite State Machines with Datapath (FSMDs) M_1 and M_2 are equivalent if for every path in P_1 there is an equivalent path in P_2 and vice versa
- Code transformations can make this job difficult
- Paths may be extended, and the path covers are updated accordingly
- $\{q_{1,0} \xrightarrow{x<y} q_{1,3} \simeq q_{2,0} \xrightarrow{x<y} q_{2,3}, q_{1,0} \xrightarrow{!x<y} q_{1,3} \simeq q_{2,0} \xrightarrow{!x<y} q_{2,3},$ $q_{1,3} \Longrightarrow q_{1,0} \simeq q_{2,3} \Longrightarrow$

A major challenge: Code motions across loops



A path, by definition, cannot be extended beyond a loop.

Thank you!